

Scalable Routing for Networked Sensors and Actuators

Thomas Fuhrmann

IBDS Systemarchitektur, Universität Karlsruhe (TH), Germany

thomas.fuhrmann@ira.uka.de

Abstract—The design of efficient routing protocols for ad hoc and sensor networks is challenging for several reasons: Physical network topology is random. Nodes have limited computation and memory capabilities. Energy and bisection bandwidth are scarce. Furthermore, in most settings, the lack of centralized components leaves all network control tasks to the nodes acting as decentralized peers.

In this paper, we present a novel routing algorithm, *scalable source routing* (SSR), which is capable of memory and message efficient routing in large random networks. A guiding example is a community of 'digital homes' where smart sensors and actuators are installed by laypersons. Such networks combine wireless ad-hoc and infrastructure networks, and lack a well-crafted network topology. Typically, the nodes do not have sufficient processing and memory resources to perform sophisticated routing algorithms. Flooding on the other hand is too bandwidth-consuming in the envisaged large-scale networks.

SSR is a fully self-organizing routing protocol for such scenarios. It creates a virtual ring that links all nodes via predecessor/successor source routes. Additionally, each node possesses $O(\log N)$ short-cut source routes to nodes in exponentially increasing virtual ring distance. Like with the Chord overlay network, this ensures full connectivity within the network. Moreover, it provides a routing semantic which can efficiently support indirection schemes like i3. Memory and message efficiency are achieved by the introduction of a route cache together with a set of path manipulation rules that allow to produce near-to-optimal paths.

Keywords: Routing, Integration of ad hoc and infrastructure networks, Topology management

I. INTRODUCTION

Routing is a well-studied topic in networking. It is the task of picking a sequence of nodes between the source that sends the packet into the network and the packet's destination. Optimally, this sequence should denote a shortest path, but a (slight) deterioration is tolerable for most applications.

One might think that routing is fully understood and that only minor improvements are conceivable. But the advent of sensor networks has shifted the requirements for routing algorithms towards very large systems of devices with limited capabilities. Often, these devices are expected to employ self-organization techniques that avoid the need for centralized components and administration. These shifted requirements lead to novel solutions that significantly differ from the traditional approaches, both in wired networks and wireless ad-hoc networks.

In this paper we discuss a novel approach to the problem of efficient routing in sensor-actuator networks, *scalable source routing* (SSR). We show how it differs from traditional routing mechanisms and point out where it creates benefit for very large networks of embedded nodes. We particularly target scenarios where ad hoc and infrastructure networks are combined, but the infrastructure is not well-structured. Guiding example is the community of 'digital homes' where sensor data is shared not only within one house but also across a whole region.

SSR aims especially at scenarios where the communication demands *do not follow* a simple, centralized pattern, but a fully distributed scenario where each node might eventually communicate with any other node in the network. Moreover, SSR maps – in a well-defined way – the entire address space to the set of actually present nodes. Such a semantic is very similar to that offered by *distributed hash table* overlay networks like e. g. Chord [1], Pastry [2], or Overnet/Kademlia [3]. The Internet indirection infrastructure (i3) [4] describes several services (e. g. multicast, anycast, concast, etc.) that can be composed out of such a semantic. Thereby, SSR efficiently supports proximity-aware data aggregation and distribution.

This paper is structured as follows: Section II briefly describes the application scenario that guided the design of the the protocol. Section III gives an overview of

classical routing algorithms and argues that they are limited by a memory – message efficiency trade-off when the networks lack a well-crafted hierarchical structure. Section IV briefly describes the novel *scalable source routing* approach that we have proposed recently. Section V evaluates this approach with respect to routing efficiency. Section VI briefly discusses related work, and section VII concludes with an outlook to future work.

II. APPLICATION SCENARIO

SSR is a protocol that can be applied in a wide area of different usage scenarios. It is especially useful for scenarios where nodes bear random addresses and/or the physical network topology does not follow any structured pattern, since there, classical approaches fail: Infrastructure network routing protocols typically create routing tables and are thus not memory-efficiently applicable when a network lacks a hierarchical network structure (cf. sec. III). Mobile ad-hoc network (MANET) and sensor network routing protocols, on the other hand, typically assume the network graph to be a (generalization of a) unit disk graph. Often, they can thus not benefit from a combination of ad-hoc and infrastructure network topology, a scenario which we believe to be of increasing importance in the near future. We will illustrate this with a use case example that guided the design of SSR, the *community of 'digital homes'*.

Imagine Jane and John Smith moving to their new home. They install quite a few technical 'gadgets', some wired, some wireless: The marquee on the veranda and the blinds throughout the house are controlled by wired switches. Most lights are connected to ordinary wall-mounted switches, but they also have a wireless remote control to dim the lights in the living-room. The wind sensor on the roof that retracts the marquee in case of an approaching storm is wired, but the temperature and humidity sensors Jane uses to monitor the conditions in her small green house in the garden are wirelessly connected.

This list could be continued almost endlessly, leading to a large conglomerate of wired and wireless links. Our vision is to provide a routing mechanism that is capable of routing messages through such a 'random' network topology. (We do not discuss any other aspects besides routing, here. Clearly, link layer protocols, security, application programming interfaces, etc. are important issues that need to be solved to make our vision finally come true. But all this is beyond the scope of this paper.) Having such a routing means, the following applications become feasible:

1. Temperature readings in the green house are picked up by a base station there and displayed wherever Jane puts up the according monitor (e.g. a small display in the living room, a wrist-watch, PDA, mobile phone, or Jane's bed-side alarm clock). The wireless signal does not need to span the entire distance as is typically the case today. The data could be relayed by the wired home infrastructure. This increases the range of applicability, reduces interference, and saves energy in the wireless end-devices.

2. Switches, lights, blinds, etc. can be arbitrarily combined. The association of a wall-mounted switch with a light is no longer fixed by physical cabling, but Jane and John could program their house according to their preferences. Wireless remote controls can be easily integrated. Since the system is self-organizing, they can arbitrarily add and remove devices without breaking the system.

3. Wind sensor readings can be aggregated¹ in the entire community to allow more reliability. Even if sensors at individual homes might miss an upcoming storm, the system is able to provide enough data to trigger retraction of the marquees.

4. Fridges and air-conditioning in the neighborhood can communicate to balance power consumption. Thereby, they avoid local or regional capacity overload in the power grid. Moreover, service personnel could monitor the systems remotely.

Again, these are only very few examples for the plentitude of possible applications altogether making up our vision. Many more could be added, especially those that benefit from linking all the houses in a community. Yet these few examples already demonstrate the potential of a routing protocol that is able to efficiently operate in random networks of embedded sensor-actuator devices.

III. THE PROBLEMS OF ROUTING

At first sight it may seem that routing is so-well studied that there is no unexplored point left in the design space. This notion is wrong, since embedded sensor networks, especially in the scenario described above, have differing requirements. We demonstrate this by re-considering the task of routing: *Routing has to produce a path of nodes through a network. It has to do so for any pair of source-destination nodes.* The Dijkstra or Bellman-Ford algorithms are e.g. well-known solutions to that problem. But even simple approaches like flooding will do, and have been used, e.g. with source routing

¹SSR routing semantics efficiently supports indirection schemes like i3 that provide data aggregation and dissemination.

bridges [5, chapter 4.4] that flood discovery frames to create source routes.

In spite of their general feasibility, all these approaches have drawbacks: Performing the Dijkstra algorithm (as with link state routing) requires significant amounts of both, processing power and memory (to store the network graph). The typical Bellman-Ford implementation (distance vector routing, DVR) requires the memory to be large enough to at least store the routing table. Furthermore, it performs badly in case of node or link churn (i.e. links that break due to nodes leaving the network or being mobile). Flooding as employed e.g. with ad-hoc routing algorithms like the *Ad hoc On-Demand Distance Vector (AODV)* routing creates even more messages than DVR.

All this renders these approaches not very well suited for embedded sensor-actuator networks, where the nodes do not have the resources to perform the Dijkstra algorithm and flooding is prohibitively expensive with respect to bandwidth and energy consumption. What is needed here is a *fully decentralized* algorithm, that is *both memory and message efficient*. The former (decentralized operation) is a typical design goal of most routing algorithms. (We mention it only to prevent critiques from claiming centralization as cure to the problem we will discuss shortly.) The latter is the part that is really hard to achieve. To see this, we have to briefly discuss where and why established approaches fail to accomplish the combined goal of memory and message efficiency. Depending on the personal point of view, either memory or message efficiency seem to be the root cause of the problem. We will therefore discuss both points of view, here.

A. The problem with source routing bridges

Put briefly, a source routing bridge maintains a cache with source routes. Upon a cache hit, the cached source route is sufficient to route the packet² to its destination.

Upon a cache miss, a discovery packet that is flooded within the network records the source route which can then be entered into the cache. This mechanism is also the foundation of many ad-hoc routing protocols.

The core problem here is message efficiency, since for each cache miss, the whole network has to be flooded. (Note that we assume the general case where nodes are equally likely to be the destination of a packet. Routing becomes of course more easy if further assumptions can

be made.) Provided the network is sufficiently stable (i.e. none or only few nodes are mobile, which is typically assumed in sensor networks), increasing the cache size could reduce the amount of discovery packets. But even in a fully stable network with node caches that can hold source routes to all the other nodes in the network, each node would have to flood the network once. This leads to $O(N)$ messages that have to be transmitted per node.

B. The problem with shortest path routing

Routing protocols, be it link state or distance vector protocols, improve the message efficiency by aggregating topological knowledge. A node does not need to cache full source routes to the potential destinations, but only know a direct neighbor that is closer to the destination than itself.

Consider e.g. distance vector routing. There, the nodes exchange their routing tables that contain the essence of all previously received routing information updates. This reduces both the size of each entry and the number of messages. But unlike with the source routing bridge example, here, the routers have to have knowledge about all the nodes of the network, not only a small subset they chose to cache. This leads to $O(N)$ routing table entries.

The Internet could not have grown to its current extent if there was no way to escape this problem. The respective solution is the introduction of hierarchies. By assigning network addresses that reflect the topology of the network, routers can aggregate the information in their routing tables.

Two aspects render this solution unsuitable for sensor networks: (1) Node addresses have to be assigned in concordance with the node's position in the network. (The severeness of this restriction depends on the concrete details of the sensor networks. A discussion is beyond the scope of this paper.) And more importantly, (2) addresses within the network have to reflect the network topology. This is only practical for certain classes of topologies. If e.g. the network was a tree, each node would have to store only the address ranges of its children and use its parent as default route. The larger the deviation from the tree topology, the larger the routing table has to be. (Some approaches artificially create a tree topology by not using all available links. This has the drawback of unnecessarily concentrating the traffic on the remaining links.) Sensor-actuator networks can be expected to have somehow 'random' topologies that are far from being a tree and thus lead to the $O(N)$ memory requirement at the nodes.

²We employ the term packet to achieve consistent wording in our reasoning, although with bridges 'frame' would be the usual term. For our purposes both concepts are equivalent.

In the following section we will briefly present our novel approach, *scalable source routing*, that breaks the trade-off just described and increases both memory and message efficiency beyond the $O(N)$ limit, even without requiring the nodes to adhere to a defined addressing scheme.

IV. A NOVEL ROUTING APPROACH

The discussion in the previous section has shown that two problems must be solved to achieve memory and message efficient routing in networks that lack a well-crafted structure:

- Avoid storing complete state by using source routes.
- Avoid flooding when acquiring source routes.

SSR uses two distinct but closely interwoven mechanisms to achieve these goals. It is inspired by the combination of distributed hash table (DHT) routing with classical source routing.

DHTs like e.g. Chord [1] maintain $O(\log N)$ routing state information at each node to reach any node of the network within $O(\log N)$ hops. The prescription specifying which node stores which routing state guarantees global consistency of the DHT.

Classically, DHTs are realized as overlay networks that use transport layer links to reach the prescribed nodes. But the mechanism carries over to the realization as a network layer protocol that uses source routes [6][7]. In that case it is necessary to establish these routes efficiently.

Two different approaches lend themselves for that important step: (1) The mechanism proposed in [8] is inspired by distance vector routing. Each node disseminates a table with source routes to its neighbors who then pick suitable source routes from the received table to optimize their own table. (2) The iterative successor pointer rewiring protocol (ISPRP) [9] establishes a virtual ring where each node maintains source routes to its successor in this virtual ring. It can be shown [10] that ISPRP converges into a globally consistent state. Moreover, it is very message efficient and almost completely avoids flooding³.

³ISPRP needs a few nodes to flood (parts of) the network during bootstrapping to resolve potential inconsistencies in the virtual ring. (See [10] for details.) In the simulations presented in this paper, inconsistencies were only found in the unit disk graph example. Accordingly, only there the flooding variant was used. Since only very few nodes actually flood the network, the according overhead was on total only 3.7 messages per node.

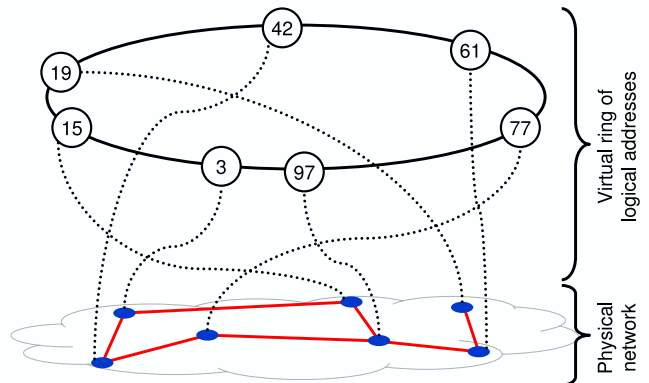


Fig. 1. The virtual ring of logical addresses

The following protocol description focuses on ISPRP as means for bootstrapping the source routes. We briefly review it in section IV-A. In order to create a full routing protocol, the scalable source routing protocol, ISPRP is enhanced by three further mechanisms besides bootstrapping:

- 1) Rules for *discovering source routes* to arbitrary destinations (sec. IV-B).
- 2) Rules for managing a source route cache in which nodes can *efficiently store their share of system's routing state information* (sec. IV-C).
- 3) Rules for source route pruning that *keep the source routes short* (sec. IV-D).

The full SSR protocol specification is available as technical report [11]. The source code of our implementation is available upon request.

A. Bootstrapping

Each node is assumed to bear a globally unique address.⁴ This could be a manufacturer assigned address, the hash of a public cryptographic key, etc. This address identifies the node's position in a virtual ring (cf. fig. 1). This ring is assumed to have a defined orientation. Note that the ring will typically be sparsely populated. Note furthermore that with respect to network topology, the address is completely random.

1. Upon deployment and initialization, a node inquires a list of the nodes that are its direct physical neighbors. This discovery mechanism depends on the details of the

⁴We do not deal with the duplicate address problem, here. It is closely linked to the problems with node churn, mobile nodes and general security aspects. All this is addressed in ongoing work. We only briefly mention that the fact that our protocol inherently provides source routes helps to reveal the inconsistencies caused by the aforementioned problems.

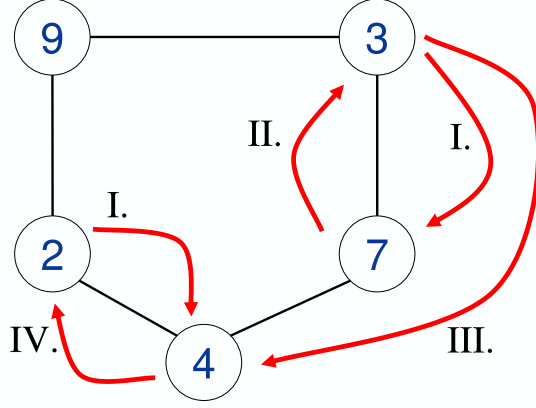


Fig. 2. I. Initial successor notifications, II. successor update: 7 notifies 3 that 4 is a better successor, III. successor notification, IV. successor update: 4 notifies 2 that 3 is a better successor, ...

physical link (wired or wireless, point-to-point or shared medium, etc.) and is thus outside the scope of this protocol. Note that, for the purpose of this paper, we assume the link layer protocol to provide a reliable bidirectional link. (This is no limitation on the applicability of SSR in wireless sensor networks, even though there links are often asymmetric and lossy. It merely means that SSR needs to be accompanied by a protocol that deals with these characteristics where they are present.)

Being direct neighbors by definition, a node can trivially cache the source route to each of its neighbors.

2. Each node then selects that node from its cache that is its presumed successor in the virtual ring. A node B is the *presumed successor* of A if no node is known to A whose address is between that of A and B (with respect to the orientation of the ring). A sends B a *successor notification* message. This can be done since by definition, A can retrieve a source route to B from its cache.

3. Upon reception of a *successor notification* message, each node can discover a potential inconsistency. In such a case a node replies with a *successor update* message containing a source route to a proposed better successor (see fig. 2 and [11] for details).

As a result of the bootstrapping, all nodes have a source route to their respective successor in their cache. (Note that the brief description given here does not handle all required protocol actions.)

The term bootstrapping indicates that ISPRP does not rely on any a priori configuration, etc. It does *not* imply that ISPRP would *only* be applicable once. To the contrary, if a new node arrives, ISPRP is able to integrate that node at its correct position on the virtual ring. The

same holds when a source route breaks e.g. due to node failure. Then ISPRP closes the gap and restores global correctness.

B. Routing

Even with the source routes to their respective successors alone, the system would be able to consistently route messages. Each message would simply be passed to the respective successor (using the cached source route) until it had reached its destination.

Clearly, this is by far not efficient. To improve the routing efficiency, each node will need to cache further source routes. Assume that each node had source routes to $O(\log N)$ other nodes, where the spanned address space distance to these nodes increases exponentially. Then any node can reach any other node in $O(\log N)$ steps. (This is the mechanism underlying the Chord DHT [1]. We combine it with the proximity route selection mechanism described in [12], and apply it to source routes similarly to an idea proposed in [6][7].)

The according rule is as follows:

4. If applicable, packets are forwarded using the source route that is contained in that packet. If the contained source route has been fully pursued and the message has *not yet* reached its destination, the respective node looks up another source route (see below) from its own cache, appends it to the packet's source route and resumes the regular source routing.

More detailedly, if a node A that needs to forward a packet towards the destination C already has a source route to that destination in its cache, it appends it to the route already contained in the packet. (At the originating node this means appending the new route to an 'empty route'.) Otherwise it chooses a route to a node B , so that

- (A, B, C) is in correct order with respect to routing direction⁵,
- the length of the source route is minimal, and
- the address space distance (A, B) is maximal.

These criteria apply in descending order, i. e. if there is a minimal cost source route to a node that is correct with respect to routing direction, the address space distance does not matter. If two (or more) source routes have equal cost (i. e. equal number of physical hops), the one that advances farther in the address space ring is chosen.

⁵Chord routes only forward with respect to the virtual ring's orientation. SSR can benefit from routing bidirectionally, since this enhances the self-stabilizing effect described in the following section. Nevertheless, as we show in sec. V the effect is also present with unidirectional routing.

The full specification [11] details the protocol further. It introduces *connection requests* as well as *unreachable* and *established* messages. These names reflect the fact that the routing actions described here are only necessary if a node does not already have a source route to a destination. In an application scenario like the one described in section II where the network is relatively stable, the routing traffic is thus kept very low. To reduce the message size when using established routes, source route paths can either be described by a list of node addresses or interface identifiers. The latter require significantly fewer bits in the packet header since they need to be only locally unique at a node, not globally unique.

C. The route cache

So far, we have simply assumed that each node already had the required additional source routes in its cache. In this section we briefly describe the route cache and how source routes are entered into and retrieved from the cache.

Although the implementation of the cache is independent from the SSR protocol itself, we assume here that the cache holds a fixed number of nodes. If further nodes are entered, the least recently used (LRU) nodes will be removed from the cache. (Both writing and reading a route are counted as 'usage' of the respectively contained nodes.)

SSR requires the cache of a node A to only operate on routes originating at A . Hence the nodes in the cache form a tree with A as root. Since cache access is such that routes are read and written from leaf to root, the LRU policy guarantees that only leaf nodes are removed from the cache. These rules facilitate an efficient implementation of the cache.

SSR requires the cache to provide the following functionality:

- Insert a source route path.
- Retrieve a route towards a given address.
- Protect a route from being removed.
- Get the address succeeding a given address.

When retrieving a route towards an address that is not in the cache, the routing rules stated above are applied. In other words, a 'retrieve' call fails only if the destination lies between the node and its successor (when routing forward in ring direction). Then the destination is termed *unreachable* and a respective notification message is

sent back⁶. (If there was any route to forward the request consistently, it would contradict the definition of 'successor'.)

But even when the 'retrieve' call is successful, the retrieved route might not end at the requested address. Then the node at the end of the so constructed source route path will have to append another route. Similarly, the cache always provides a successor address to a given address, although this reflects only the limited view of the cache at the respective moment.

Route protection is needed to ensure that the routes to both the successor and predecessor nodes are always maintained in the cache. But this means also that the cache can overflow. If that happens, SSR is no longer able to consistently route all addresses (just like during bootstrapping). If the overflow is transient, SSR will recover consistency. But the cache must in any case be large enough that such an overflow is a rare event, i. e. the cache should be able to hold significantly more nodes than needed for a route that spans twice the network diameter.

Besides the successor and predecessor routes, the routes conveyed by connection requests are inserted into the cache, too. This is done so that the cache benefits if the newly inserted path contains short-cuts for routes already in the cache, but avoids deterioration of the routes in the cache in the opposite case. To avoid whole sub-trees from being lost if a link in the network breaks, the route cache should be supplemented by a link cache that stores links that are not entered into the route cache since they are suboptimal.

As we will show in section V, inserting all conveyed routes into the route cache leads to the cache automatically accumulating those routes that create the good scaling behavior known from Chord. The reason is that if a node happens to acquire a useful route, the LRU rule keeps it in the cache. Since requests for a certain address are likely to be sent to a small set of nodes only (namely those close to that address), these nodes fill their cache with the respective source routes. Altogether, this creates a self-stabilizing effect.

D. Source route pruning

To avoid inefficiently long source routes that would result from the routing process as described so far, routes

⁶The full protocol provides an additional routing semantic, where the last hop processes the message in such a case. This is e. g. employed when providing indirection functionality. A detailed discussion of this application semantics is beyond the scope of this paper.

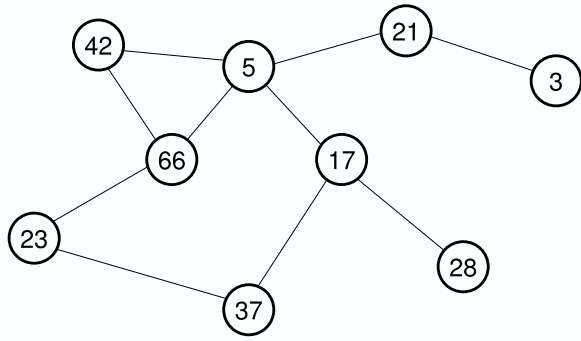


Fig. 3. Illustration for the source route pruning example

are pruned after concatenation. This applies to successor updates and connection requests. If the cache is able to provide short-cuts for the incoming source route path, these will also be taken into account.

Consider the situation depicted in fig. 3. Assume node #3 wants to create a source route to node #42. According to the routing rules, it sends a connection request to #21 which appends a route to #23 (its successor). #23 appends a one hop route to node #37 which can append the remain route directly to #42. Instead of sending a packet containing the entire route (3-21-5-66-23-37-17-5-42), here, the route is pruned to 3-21-5-42. (In the simple example this happens to be a globally shortest path from #3 to #42. Although this is not generally the case, the next section provides evidence that pruning can yield quite good paths.)

Since #37 is no longer contained in the pruned routes, it creates a *container* packet that conveys the connection request to #5. The respective route (37-17-5) is called stub. When extracting a connection request from a container the stub is conversely conveyed in the connection request. The reason is that if a subsequent link is broken, the node that appended the respective source route path can be accordingly informed to remove the stale entry from its cache.

V. EVALUATION

We implemented the described protocol in C++ and simulated random graph networks with several sizes and topologies. We focused especially on topologies having the so-called *small-world* property, i.e. power-law degree distribution (see below). Such a topology is typical for infrastructure networks, both planned and organically grown, e.g. the telephone network and the Internet. In these simulations we specifically focused on two aspects: (1) How quick and message efficient does

the system converge into its globally consistent state? (2) What is the achieved path length of the source routes as compared to shortest paths that could be obtained by classical, non-hierarchical routing algorithms?

A third important aspect would have been: (3) How does the cache size influence the performance? — We studied this question, too, but found hardly any effect in our simulations. (The only exception were Erdos-Renyi random graphs which are of no practical relevance.) If the cache was large enough to allow convergence, then the achieved performance did not improve by increasing the cache. (As stated above the cache needs to hold significantly more nodes than twice the network diameter to store successor and predecessor and leave room for operating on routes conveyed in connection requests.) Clearly, the cache size can be expected to influence the system performance for very large networks. But up to the limit⁷ of our simulation machine (128 000 nodes) we could not produce this effect.

A. Simulation setup

Each node repeatedly issued connection requests for a node that was drawn randomly (with uniform distribution) from the set of all nodes. (More detailedly: Each time when either a source route was established or when a node had reported that it could not further forward the request, a new connection request was issued.) It was then logged whether the an according source route could be established or not. If a source route was found, the ratio of its length to the network's shortest path⁸ between the two nodes was logged. In the following, we call this parameter the achieved *routing stretch*. The fraction of unsuccessful connection attempts was taken as an indicator how far the bootstrapping had proceeded with convergence towards global consistency.

B. Global consistency and achieved routing stretch

Figure 4 shows the results for a 8000 and a 128 000 node random graph network with power-law degree distribution. Such networks are often termed small-world networks. In both cases each node's source route cache contains only 255 other nodes. (In our implementation, this corresponds to 4 KB memory which is suitable for sensor networks.)

⁷The limit is caused by the comparatively bad scaling of the Dijkstra algorithm with which we compared our algorithm. It is *not* caused by our protocol. This could e.g. be seen when profiling the CPU usage of our simulation.

⁸The network's shortest path is the path obtained with the Dijkstra algorithm using full knowledge of the network topology.

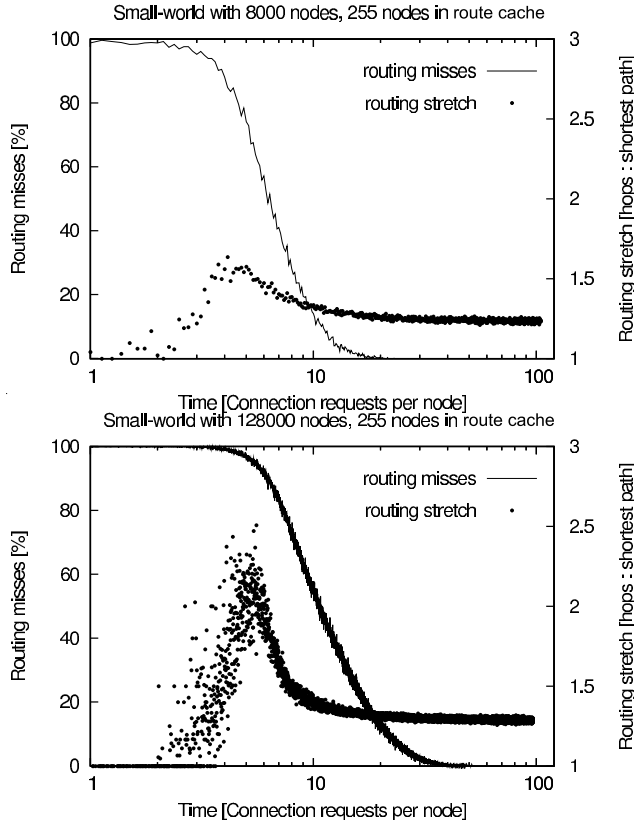


Fig. 4. Convergence and achieved routing stretch

Both figures show the number of routing misses as line (scale on the left hand side) and the achieved routing stretch as dots (scale on the right hand side). Each value is an average of 1000 exchanged messages.

Immediately after start-up the network is inconsistent and almost all connection requests fail. Only the rare requests for addresses in the vicinity of a node can be positively answered. The resulting source routes are typically shortest paths. Hence the low routing stretch at the beginning.

After each node has issued about 3–4 connection requests, the 8000 node network is about to converge⁹ (fig. 4, upper graph). At that time enough successor updates have been exchanged (not shown in the figure) to provide successor consistency for a substantial fraction of the nodes.

Although now more and more connections can be

⁹Convergence here means that the virtual ring is becoming consistent so that more and more connection requests can be responded correctly. See [9] for a proof that unpartitioned networks always converge in the absence of node churn. Note that even before convergence the network is already functional, even though some connections cannot yet be established.

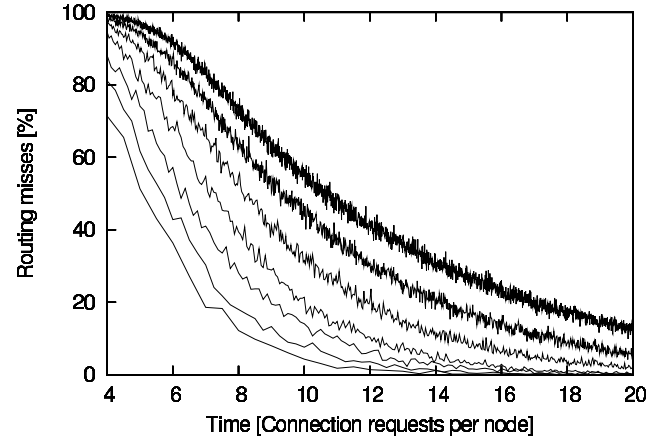


Fig. 5. Convergence for network sizes from 2000 to 128 000 nodes

established, the achieved routing stretch becomes worse and worse peaking at a value of more than 1.5. Then the knowledge about good paths builds up in the path stores and the routing stretch becomes better and better, finally stabilizing at about 1.2. At that time, each node in the network has on average exchanged 5.5 successor notification messages, 15% of which were incorrect i.e. caused an immediate subsequent successor update (stable network, no packet loss).

The lower graph in figure 4 shows a network which is significantly larger. Here, convergence sets on later and takes much longer to prevail. Moreover, at the onset of convergence the routing stretch peaks at a significantly higher value of about 2.5. Nevertheless, the routing stretch drops again quickly, finally stabilizing at about 1.3.

Figure 5 shows the convergence for network sizes from 2000 to 128 000 nodes. From left to right the curves correspond to a stepwise doubling of the network size. Although the curves fluctuate erratically due to the randomness contained in our simulation, one nevertheless observes that at each ordinate the curves are almost equidistant. This corresponds to an $O(\log N)$ scaling.

In accordance to theoretical studies from the field of peer-to-peer networks and random graphs [12][13] we expect the achievable routing stretch to scale with $O(\log(\log N))$. As explained above, we could not push our simulations far enough to actually test this, but we can already conclude that our protocol scales well even for very large networks of 100 000s of nodes.

C. Other topologies

Although we recommend our protocol for networks with power-law node degree distribution (small-world

networks), we studied its performance for other topologies, too. Figure 6 shows the results for a unit disk graph, with 8000 nodes uniformly randomly distributed on a 40×40 square. (Nodes are linked if their Euclidian distance is ≤ 1 . Hence the name *unit disk graph*.) We explicitly avoided network partitioning by adding links to produce a connected graph if necessary. Given the parameters used here, we found that this to occur only rarely, but it ensures that the results are comparable to the other scenarios discussed in this paper.

Note that we only show the case with 511 nodes in the route cache. The reason is that with unit disk graphs, path lengths are on average so large (about 30 hops here), that a smaller cache is insufficient to allow convergence. (Remember that all nodes need to hold source routes to their successor and predecessor. Furthermore, they need to temporarily store the paths contained in connection requests so that they can perform the respective operations.) Moreover, as described above, we needed to activate the flooding option to avoid potential inconsistencies. Yet, since only very few nodes actually do flood the network, the associated overhead is negligible. (On average each node had to process 3.7 flooding messages during bootstrapping.)

Despite this need for a considerably larger route cache, SSR performs well in this scenario, too. Since even the shortest paths are already very long, SSR's relative overhead is almost negligible, leading to a stretch of only about 1.1. Although here, no hub nodes are present in the graph, it is – in a moderately dense graph – highly likely that intersecting¹⁰ paths will have common nodes.

If we increase the node density, this probability drops and SSR is no longer able to efficiently cut down the paths.

In a sensor-actuator network, however, such a high density is unnatural, and thus this property is no severe limitation. Either, there is an additional infrastructure network with hub nodes present, or the density of *active* nodes should be low anyway. (Note that otherwise, many nodes would contend for the medium and thus waste energy.)

Figure 7 shows the results for an 8000 node Erdos-Renyi random graph. Such graphs present a worst-case scenario since there are no hub nodes but many potential short-cuts.¹¹ Full topology routing algorithms find all

¹⁰Here, 'intersecting' alludes to the idea of a path as geometrical line joining the respective nodes.

¹¹In Erdos-Renyi graphs all pairs of nodes have equal probability to be connected by an edge.

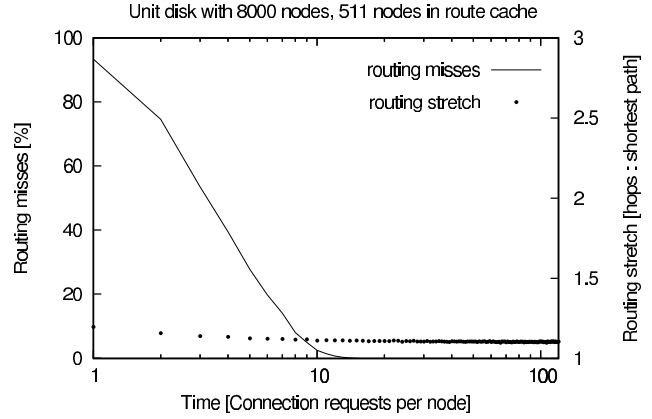


Fig. 6. Convergence and achieved routing stretch for unit disk random graphs

these short-cuts at the expense of acquiring and storing the entire network graph.

As one can see from the figure, SSR is capable of routing in such an extreme topology, but it misses out the short-cuts. Hence the considerably large routing stretch of 2.2. When we double the size of the route cache, the stretch drops to 1.7. Further increases of the cache size bring the stretch further down. When the cache can hold the entire network graph (as is the case with full topology routing algorithms), the stretch reaches expectedly about 1.0 in the long run.

The nature of these two extremes is further illustrated in figure 8. The plots show the cumulated distribution functions for the source route path lengths produced by SSR and a full-topology algorithm (Dijkstra) in the three scenarios described above.

In the small-world case, SSR is capable of finding near-to-optimal paths for almost all the nodes in the network: For nodes with distances of 4-5 hops, SSR produces paths that are on average only up to one hop longer than the optimum. For nodes with larger distances, SSR's overhead is 1-2 additional hops.

The Erdos-Renyi case demonstrates, how SSR misses out the short-cuts hidden in that extreme network topology. Even though nodes are again only up to 10 hops apart (=shortest paths), SSR produces paths of up to 30 hops. (Note that this case is of academic nature only. Actual networks do *not* have such a topology!)

In the unit disk scenario, SSR is again able to produce near-to-optimal paths. Most paths are only 1-2 hops longer than the shortest paths, but according to the structure of unit disk graphs, paths are much longer than in the other two scenarios (about 30 hops compared to only 5 hops in the small-world case). This leads to the

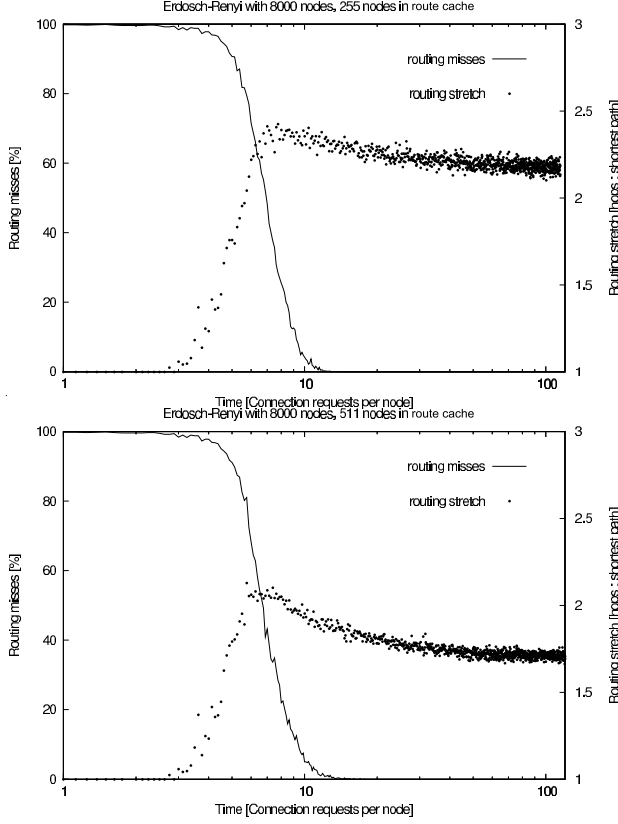


Fig. 7. Convergence and achieved routing stretch for Erdos-Renyi random graphs

undesirable effect that the route cache needs to be much larger than with the small-world scenario.

D. Route cache structure

Chord's routing efficiency is based on the fact that its routing table entries (there called 'fingers') span exponentially increasing distances in the virtual address ring. Figure 9 demonstrates that this efficient distribution is an *emergent* feature of our protocol, too.

To this end, we calculated for each node the address distance between its address and all the addresses in its route cache (16 000 node small-world network) and binned the results into 320 bins with exponentially growing size. (I.e. the i th bin contained the distances between $2^{i/10-1}$ and $2^{i/10} - 1$.)

At small distances, i.e. between the expected average successor distance $2^{32}/16\,000 \simeq 250\,000$ and 10^7 , the system actually manages to obtain the optimal Chord-like distribution (flat line). Above 10^7 the system follows the uniform distribution (slanted line) that would result from the nodes storing arbitrary other nodes. This excess is caused by the fact that unlike with Chord our protocol

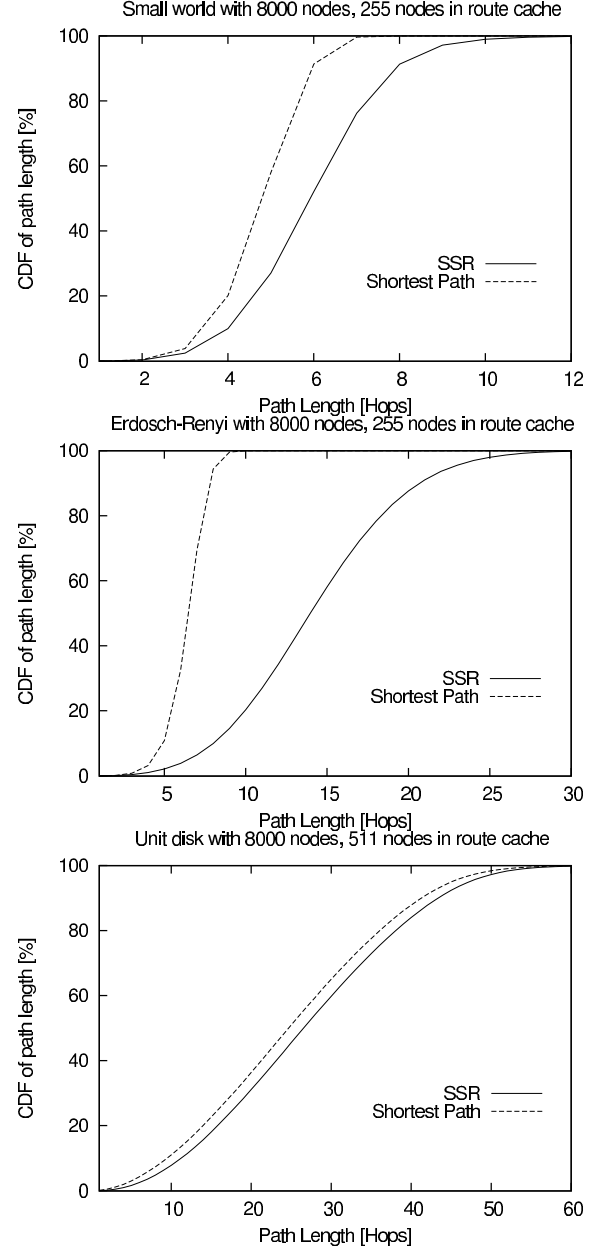


Fig. 8. CDF of the routing stretches

has to store source routes that, by definition of a random network, contain random addresses. (Note that close inspection shows that the accumulation at small distances corresponds to a slight depletion at large distances.) Only at very large distances the distribution exceeds again the uniform distribution. This is caused by the nodes storing the paths to their predecessors.)

E. Traffic concentration

In our simulations we mainly used a random graph model that had the so-called small-world property, i. e. in

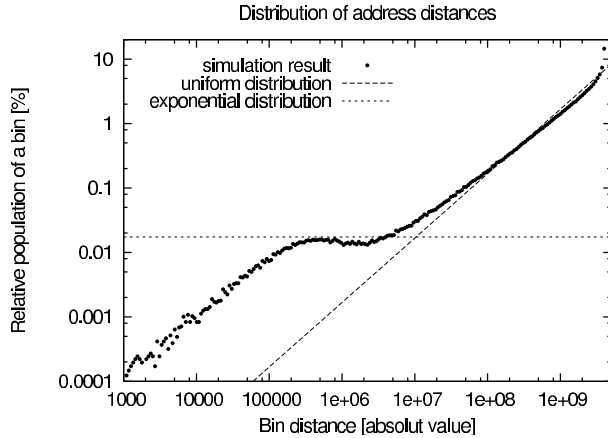


Fig. 9. Distribution of node distances in the path store

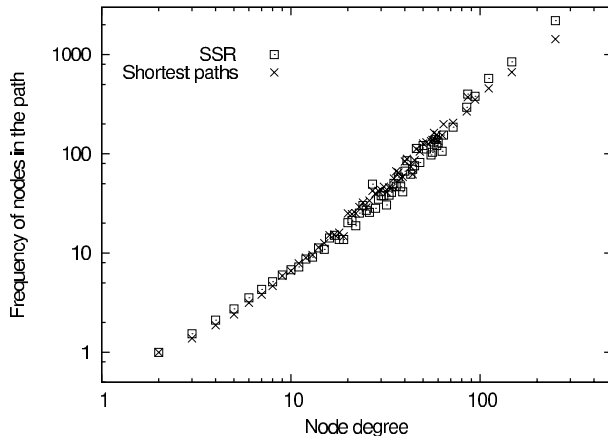


Fig. 10. Frequency of nodes in the path

which the node degrees had a power-law distribution. Such networks contain so-called *hub* nodes [14]. These serve as natural landmarks and enable the nodes to efficiently prune the source route paths: If two paths both happen to contain the same two hub nodes, both paths can share their knowledge about a good path between these two hub nodes.

One might think that this leads to a traffic concentration at the hubs since one might believe that detours containing hubs are likely to accumulate in the route caches. However, our simulations show that the latter is *not the case*. We compared the frequency of the nodes in the route caches (16 000 node small-world network) with their frequency in the shortest paths in that network. Both frequency distributions follow almost the same power-law distribution: $k^{1.6509}$ in the shortest path case and $k^{1.6335}$ with our SSR protocol. I.e. within the simulation errors there is no increased preference for hub nodes (cf. fig. 10).

VI. RELATED WORK

Recently, various authors have proposed combinations of distributed hash table (DHT) overlay routing with mobile ad-hoc network routing protocols in the network layer. Most of these proposals treat overlay and network layer as completely separated, and are thus not comparable to our approach. Although we are not aware of detailed studies, it seems evident to us that this approach leads to a considerable traffic overhead and is thus not suitable for sensor-actuator networks.

Other authors follow an approach that is closer related to our work. Like us, they propose to employ peer-to-peer routing techniques to build a network-layer routing protocol [15][16]. In contrast to the virtual ring established by our protocol, their approach is based on hierarchical, tree-based routing. Moreover, they aim at the Internet and do not consider sensor-actuator networks with their limited capability nodes, where protocol complexity is an important performance aspect. Since their design is not yet complete and therefore, no experimental performance evaluation is available, we cannot compare it to our approach.

Viana et al. propose a routing protocol, *Tribe*, that assigns node addresses hierarchically according to the network's topology [17]. Each node bears three identifiers which have to be resolved successively using a distributed hash table. We believe that *Tribe*'s design, too, is more complex than ours. Since it aims at mobile ad-hoc networks, the authors study unit disk graph topologies only. There, the achieved path lengths are very similar to ours, but unlike SSR, *Tribe* creates a severe traffic concentration at the root node of the hierarchy. Moreover, it remains an open question how *Tribe* performs in other scenarios, especially in the small-world topologies.

Hu et al. propose to use the Pastry overlay for unicast routing in ad-hoc networks, where routes between virtual addresses are discovered using the DSR routing protocol [6][7]. Their work differs from ours in that SSR does not require all nodes to flood the network in order to discover routes. Thus SSR produces a significantly lower overhead.

VII. CONCLUSION AND OUTLOOK TO FUTURE WORK

In this paper we have presented a novel routing mechanism, *scalable source routing*, which is inspired by distributed hash table routing protocols. Unlike classical routing algorithms that either flood the network to obtain

source routes or have to maintain potentially large routing tables, our protocol distributes the topological knowledge that is required for routing among all the nodes in the network. Thereby, both the memory requirement at each node and the communication overhead are very low.

With the help of simulations we have demonstrated that our protocol is nevertheless capable of achieving a routing stretch of only about 1.2 to 1.3 for large small-world random networks containing nodes with random addresses. The latter is a unique feature that distinguishes our protocol from state-of-the-art routing protocols that require node addresses to follow some structured scheme in order to be scalable.

This ability to *efficiently* work in random environments makes our protocol especially suitable for networks that are not centrally planned or managed, and that thus do not obey strong hierarchies. Its low resource demands recommend our protocol for sensor-actuator networks where lots of small nodes need to communicate in a self-organizing manner.

Currently, we study extensions to the basic SSR protocol described so far. These will increase the performance of the protocol in the presence of mobile nodes and, in general, node and link churn. Moreover, we investigate means to secure the protocol from malicious nodes injecting wrong sources routes into the network. We believe that with the help of these extensions, SSR bears a great potential for extending its application area beyond the scenario underlying this paper.

ACKNOWLEDGMENTS

This work was supported by Deutsche Forschungsgemeinschaft under grant FU448-1. Special thanks to Kendy Kutzner and Curt Cramer for the valuable discussions and their support with the simulation machines.

REFERENCES

- [1] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications", in *Proceedings of the SIGCOMM 2001 conference*. 2001, pp. 149–160, ACM Press.
- [2] A. Rowstron and P. Druschel, "Pastry: Scalable, distributed object location and routing for large-scale peer-to-peer systems", in *Proceedings of the IFIP/ACM International Conference on Distributed Systems Platforms (Middleware) 2001*, Heidelberg, Germany, Nov. 2001.
- [3] P. Maymounkov and D. Mazières, "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric", in *Revised Papers from the First International Workshop on Peer-to-Peer Systems*. 2002, pp. 53–65, Springer-Verlag.
- [4] I. Stoica, D. Adkins, S. Zhuang, S. Shenker, and S. Surana, "Internet Indirection Infrastructure", in *Proceedings of the 2002 ACM SIGCOMM conference*, Pittsburgh, PA, USA, 2002, pp. 73–86, ACM Press.
- [5] A. S. Tanenbaum, *Computer Networks*, Prentice Hall, 2002.
- [6] Y. C. Hu, S. M. Das, and H. Pucha, "Exploiting the Synergy between Peer-to-Peer and Mobile Ad Hoc Networks", in *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, 2003, pp. 37–42.
- [7] H. Pucha, S. M. Das, and Y. C. Hu, "Ekta: An Efficient DHT Substrate for Distributed Applications in Mobile Ad Hoc Networks", in *Proceedings of the 6th IEEE Workshop on Mobile Computing Systems and Applications (WMCSA 2004)*, English Lake District, UK, Dec. 2004.
- [8] K. Kutzner, C. Cramer, and T. Fuhrmann, "Towards Autonomic Networking using Overlay Routing Techniques", in *Proceedings of the 18th International Conference on Architecture of Computing Systems (ARCS '05) - System Aspects in Organic and Pervasive Computing*, Innsbruck, Austria, March 2005.
- [9] C. Cramer and T. Fuhrmann, "ISPRP - A Message-Efficient Protocol for Initializing Structured P2P Networks", in *Proceedings of the IEEE International Workshop on Strategies for Energy Efficiency in Ad Hoc and Sensor Networks (IEEE IWSEEASN'05)*, Phoenix, AZ, April 7-9 2005.
- [10] C. Cramer and T. Fuhrmann, "Self-Stabilizing Ring Networks on Connected Graphs", Submitted for publication, 2005.
- [11] C. Cramer, T. Fuhrmann, and K. Kutzner, "Scalable source routing - protocol specification, version 1.0", Tech. Rep. 2005-4, Faculty for Informatics, University of Karlsruhe, 2005.
- [12] K. Gummadi, R. Gummadi, S. Gribble, S. Ratnasamy, S. Shenker, and I. Stoica, "The Impact of DHT Routing Geometry on Resilience and Proximity", in *Proceedings of the SIGCOMM 2003 conference*. 2003, pp. 381–394, ACM Press.
- [13] B. Bollobas, *Random Graphs*, Cambridge University Press, 2001.
- [14] A.-L. Barabasi, *Linked: The New Science of Networks*, Perseus Publishing, 2003.
- [15] J. Eriksson, M. Faloutsos, and S. Krishnamurty, "PeerNet: Pushing Peer-to-Peer Down the Stack", in *Proceedings of the 2nd International Workshop on Peer-to-Peer Systems (IPTPS '03)*, Claremont Hotel, Berkeley, CA, USA, Feb. 2001, Springer Verlag.
- [16] B. Ford, "Unmanaged Internet Protocol", *ACM SIGCOMM Computer Communications Review*, vol. 34, no. 1, pp. 93–98, Jan. 2004.
- [17] A. C. Viana, M. D. de Amorim, and S. Fdida, "An Underlay Strategy for Indirect Routing", *Wireless Networks*, vol. 10, pp. 747–758, 2004.